An Efficient Hierarchically Parallel GPU-Accelerated Framework in Large-Scale MIMO Detection

Qinglin Xun*†, Xueting Xu*, Fan Jiang*, Ke Luo*, Jingpu Duan*, Weichao Li*✉, Jinbei Zhang*†✉, Xiaofeng Tao*‡

*Pengcheng Laboratory, China
†Sun Yat-sen University, China
‡Beijing University of Posts and Telecommunications, China

***Abstract*—Large-scale multiple-input multiple-output (MIMO) systems offer high spectral efficiency but introduce significant computational challenges compared to smaller MIMO systems. A key bottleneck is the high-dimensional matrix inversion used for detection tasks. Existing matrix inversion methods are often limited by insufficient computational efficiency and poor scalability, which compromise the achievable spectral efficiency gains in MIMO.**

In this paper, we propose ParaNeumann, a GPU-accelerated hierarchical parallel framework for near-real-time matrix inversion in large-scale MIMO detection. The core idea of ParaNeumann is to reformulate the inversion process into a three-level parallel architecture, spanning inter-subcarrier, inter-submatrix, and intra-matrix granularities to fully exploit GPU computational resources. Within this hierarchical structure, we integrate block-diagonal structures into the Neumann series to accelerate convergence, and leverage low-precision arithmetic to reduce memory cost and improve throughput. Evaluation results demonstrate that for 512×512 MIMO scenarios, ParaNeumann incurs a negligible performance loss of only 0.5 dB compared to exact matrix inversion at a Bit Error Rate (BER) of 10^{-5}. In terms of computational efficiency, our proposed framework requires as little as one-third of the computation time compared to the cuBLAS-based baseline on a commercial RTX 4090 GPU.

***Index Terms*—Large-scale MIMO detection, Matrix inversion, Neumann series approximation, GPU acceleration.**

I. Introduction

Driven by extreme-bandwidth services, 6G aims to deliver up to a $10\times$ capacity increase over 5G [1]. Achieving this gain requires higher spectral efficiency, pushing MIMO toward large-scale arrays with hundreds to thousands of antennas [2].

This massive scaling of antennas imposes a severe computational challenge in base stations (BSs). Minimum Mean Square Error (MMSE)/Zero-Forcing (ZF)-based linear detection are most widely used in practice, and their dominant cost comes from matrix inversion, whose complexity grows as $O(N^3)$ with the antenna dimension N [3]. As the number of antennas scales to hundreds or thousands in 6G scenarios, these linear methods still suffer from excessive computation burden.

To mitigate the computational burden, Graphics Processing Units (GPUs) have emerged as promising accelerators for 5G/6G distributed units [4]. With massive parallelism and high programmability, GPUs are intrinsically suited for classical physical-layer (PHY) signal processing tasks dominated by parallel linear-algebra operations. However, directly migrating PHY algorithms to GPU platforms remains non-trivial, since they still face significant challenges in terms of computational efficiency, scalability, and numerical precision as detailed below.

Challenge 1: The sequential nature of direct matrix inversion algorithms leads to low computational efficiency. Direct methods, such as LU decomposition [5], invert matrices by solving linear equations. For an $N \times N$ matrix, the algorithm entails $2N$ strictly sequential steps of intensive vector multiplications. Since each iteration depends on the previous result, this intrinsic dependency severely inhibits parallelization, causing significant efficiency degradation as antenna dimensions scale.

Challenge 2: Traditional Neumann series (NS) methods exhibit limited applicability in diverse system configurations. The NS [6]–[8] performs matrix inversion by truncating the matrix expansion series. Although this method reduces computational load, the validity of the truncation relies on strict diagonal dominance of the target matrix. Specifically, the algorithm convergence requires a BS-to-user antenna ratio ($r = N_B/N_U$) exceeding 5.8 [6]. This requirement restricts the number of supported users, which significantly degrades the spectral efficiency of massive MIMO systems.

Challenge 3: Existing matrix inversion methods exhibit low hardware efficiency on GPU. Compared to traditional hardware such as ASIC and FPGA, GPU offers high programmability and massive computational capacity, making it a promising platform for accelerating matrix operations, particularly in 6G applications [9], [10]. To fully leverage the computational advantages of GPU, it is critical to design an algorithm that makes effective use of GPU. However, traditional matrix operation algorithms primarily aim to reduce computational complexity, overlooking these practical hardware-related considerations. Therefore, it is important to design algorithms that work efficiently with the

✉ Corresponding authors: Weichao Li (liwc@pcl.ac.cn) and Jinbei Zhang (zhjinbei@mail.sysu.edu.cn).

This work was supported by the Mobile Information Networks National Science and Technology Major Project under Grant 2025ZD1303200, in part by the National Youth Talent Program, in part by the Guangdong Provincial Pearl River Talents Program under Grant 2023QN10X714, in part by the NSF China under Grant 62471505, and in part by the PCL-CMCC Foundation for Science and Innovation under Grant 2024ZY1A0010.

hardware to achieve high computational performance.

To address the above challenges, we propose *ParaNeumann*, a GPU-accelerated parallel framework for near-real-time matrix inversion in large-scale MIMO detection, which facilitates high-performance data transmission in next-generation wireless networks. To maximize GPU occupancy, ParaNeumann employs a three-level parallel architecture spanning inter-subcarrier, inter-submatrix, and intra-matrix granularities. Furthermore, it integrates block-diagonal-based Neumann-series expansion for rapid convergence, while leveraging low-precision arithmetic to optimize memory bandwidth and throughput.

• **A Hierarchically Parallel Algorithm Architecture.** To address inefficient computational architectures, ParaNeumann reformulates matrix inversion into a three-level parallel hierarchy: inter-subcarrier processing for independent tasks, inter-submatrix parallelism via block-diagonal Neumann-series, and intra-matrix acceleration using fused Matrix Multiply-Accumulate (MMA) operations. Evaluation shows that for 512×512 MIMO, ParaNeumann requires as little as one-third of the computation time compared to the cuBLAS-based LU baseline on a commercial RTX 4090 GPU.

• **A Highly Applicable Block-Diagonal Neumann Series Method.** We propose a four-block partition strategy to overcome the limited applicability of conventional methods. By capturing more energy, this block-diagonal formulation relaxes reliance on diagonal dominance and accelerates convergence. Furthermore, it exploits structural sparsity to prune zero-block multiplications, significantly reducing computational costs. Results demonstrate that ParaNeumann maintains a gap of less than 0.5 dB in BER compared to exact LU even in low-r conditions, and significantly outperforms DNS.

• **A Memory-Efficient Implementation.** We optimize the memory hierarchy by maximizing shared-memory usage and preallocating buffers to minimize latency. Additionally, based on dynamic range analysis, we integrate half-precision arithmetic to reduce bandwidth consumption. By effectively alleviating the computational burden, ParaNeumann processes a batch of 128 matrices for 512-antenna massive MIMO systems in 20 ms on a commercial RTX 4090 GPU. This performance bridges the gap between theoretical complexity and practical deployment, demonstrating strong potential for next-generation real-time systems.

Notation: Calligraphic uppercase letters (e.g., $\mathcal{W}$), bold uppercase letters (e.g., $\mathbf{W}$), bold lowercase letters (e.g., $\mathbf{w}$), and italic lowercase letters (e.g., w) denote tensors, matrices, vectors, and scalars, respectively. The superscripts $(\cdot)^H$ and $(\cdot)^{-1}$ denote the Hermitian transpose and matrix inversion, respectively. $\mathbb{E}\{\cdot\}$ represents the statistical expectation operator, while $\|\cdot\|_F$ denotes the Frobenius norm. $\mathbf{I}_N$ stands for the $N \times N$ identity matrix, and $\text{diag}(\cdot)$ constructs a block-diagonal matrix from its input. Finally, $\mathbb{C}^{M \times N}$ represents the space of $M \times N$ complex-valued matrices.

II. Related work

The evolution toward large-scale MIMO systems is a key enabler of next-generation wireless communications. By deploying a massive number of antennas at the BS, large-scale MIMO achieves significant gains in spectral and energy efficiency as well as link reliability, which are crucial for high-capacity and low-latency applications. With the growing antenna array size, the computational complexity of baseband processing increases sharply, particularly for detection and precoding. Existing solutions can be broadly categorized into two classes: nonlinear algorithms and matrix inversion-based linear algorithms. However, these approaches still struggle to simultaneously meet the following three requirements: low computational load, high computational efficiency, and high scalability, as summarized in Table I.

A. Nonlinear Method

In nonlinear detection, the sphere mecoder (SD) method achieves near-optimal performance by exhaustively exploring all possible transmitted symbol combinations, which can be represented as a tree search where each level corresponds to an antenna and each branch to a constellation symbol. The complexity of SD grows exponentially with the number of antennas and the constellation order, i.e., $O(M^N)$ with constellation size M and the number of antennas N. For instance, with $N = 512$ antennas, 4-QAM, and 512 subcarriers, the worst-case complexity reaches $4^{512} \approx 10^{308}$ candidates per subcarrier, resulting in more than 10^{311} operations when metric updates are included, which is computationally infeasible in practice. Although GPU-based implementations [11], [19]–[21] exploit tree partitioning and parallel decoding to improve efficiency, the computational burden remains prohibitive and scalability is limited to small MIMO sizes (e.g., $N \leq 8$).

The K-best detector [12] reduces SD's exponential search by retaining only the K most likely paths at each level. However, per-level sorting introduces synchronization overhead, and the required K grows rapidly with antenna count and constellation size, making it impractical for large-scale MIMO.

The fixed-complexity sphere decoder (FCSD) [13], [22] achieves constant load by expanding a predetermined number of nodes per level, avoiding SD's uncertainty and K-best's sorting bottleneck. The FlexCore [22] framework exploits its regular structure for fine-grained GPU parallelization. However, FCSD remains exponential in antenna count, with implementations restricted to 12×12 MIMO.

B. Linear Methods Based on Matrix Inversion

While nonlinear detectors can achieve near-optimal performance, their exponential complexity renders them impractical for massive MIMO deployments. Consequently, research attention has shifted toward linear detectors and precoding schemes such as MMSE and ZF, where the computation of matrix inversion becomes the primary bottleneck. To address this issue, the literature has explored several approaches to computing the matrix inverse, which can be broadly classified into direct, iterative, and approximate methods.

TABLE I: Comparison of representative detection and precoding methods.

Category	**Nonlinear**	**Linear (Matrix-inversion-based)**				
Type	—	Direct methods	Iterative methods		Approximate methods	
Method	SD [11], K-best [12], FCSD [13]	LU [5], GE [14], Cholesky [15]	GS [16]	Jacobi [17]	Neumann [6], Newton [18]	**ParaNeumann** (Proposed)
Complexity	Exponential in N[(1)]	$\mathcal{O}(N^3)$	$\mathcal{O}(N^3)$	$\mathcal{O}(N^3)$	$\mathcal{O}(N^3)$	$\mathcal{O}(N^3)$
Efficiency	Low (Tree search)[(2)]	Low (Sequential)[(3)]	Low (Sequential)	Medium	Medium	**High** (Hierarchical parallelism)
Scalability	Low (Small N)	Medium (Limited N)	Medium	Medium	Medium (Restrictive ratio r)	**High** (Large N, flexible r)

[(1)] Computational complexity grows exponentially with the number of antennas N.
[(2)] Tree-structured sequential search means that each node expansion depends on the result of the previous level, which has limited parallelism.
[(3)] Highly sequential dependencies mean that each computation step relies on the results of the previous one, which hinders parallel execution.

1) Direct Methods: Direct methods compute the exact matrix inverse through deterministic algebraic operations such as gaussian elimination (GE), Cholesky decomposition, and LU decomposition. Although these methods yield exact solutions, their inherent sequential dependencies result in limited parallelism. For instance, during the forward substitution phase of LU decomposition (solving $\mathbf{Ly} = \mathbf{b}$), calculating y_i requires the results of $y_1, \ldots, y_{i-1}$. This dependency creates a computational chain $y_1 \to y_2 \to \cdots \to y_N$ that should be executed serially. For an $N \times N$ matrix, both forward and backward substitutions require N iterations, leading to $2N$ sequential steps that constrain scalability.

To improve efficiency, several studies have investigated GPU-based implementations. For example, GE [14] and Cholesky decomposition [15] have been parallelized for computing the matrix inverse, but the reported results are restricted to relatively moderate dimensions, such as 128×16 MIMO.

2) Iterative Methods: Iterative methods, such as Jacobi and Gauss-seidel (GS), solve $\mathbf{Ax} = \mathbf{b}$ without explicitly computing the inverse. Starting from an initial guess $x^{(0)}$, they generate successive approximations until convergence.

In the Jacobi method, each component is updated solely from the previous iteration, making updates independent and naturally parallelizable, but requiring a large antenna ratio for convergence. By contrast, GS updates each component in place using the most recent values, accelerating convergence but creating sequential dependencies that reduce computational throughput.

3) Approximate Methods: Approximate inversion methods decompose the target matrix into several initial matrices and then gradually approximate the inverse using a series expansion, with higher-order terms truncated to further refine the approximation. Neumann series-based approximate algorithms, which utilize the property of channel hardening [23], have been proposed for MIMO signal detection [6]. Another closely related approximate method is the Newton iteration (NI), which can be regarded as a variant of the Neumann series. The result after P iterations in NI is equivalent to the Neumann series expansion of order $2^P - 1$ [18].

However, these methods have practical limitations. Both the NS and NI-based approximations rely heavily on the structure of the seed matrix, which is typically diagonal or diagonal-like. For such a design to be effective, the original matrix should exhibit strong diagonal dominance, meaning that the main diagonal elements dominate the off-diagonal ones. In large-scale MIMO systems, this condition is usually met only when the antenna ratio is sufficiently high. Studies have shown that reliable convergence can be achieved only when the antenna ratio exceeds 5.8 [6], [24], meaning that, for instance, a base station equipped with 1024 antennas can effectively serve fewer than 180 users. Such a high ratio severely limits the practical deployment of these methods and reduces spectral efficiency. Consequently, the applicability of approximate inversion techniques in realistic large-scale MIMO systems remains limited.

III. Preliminary

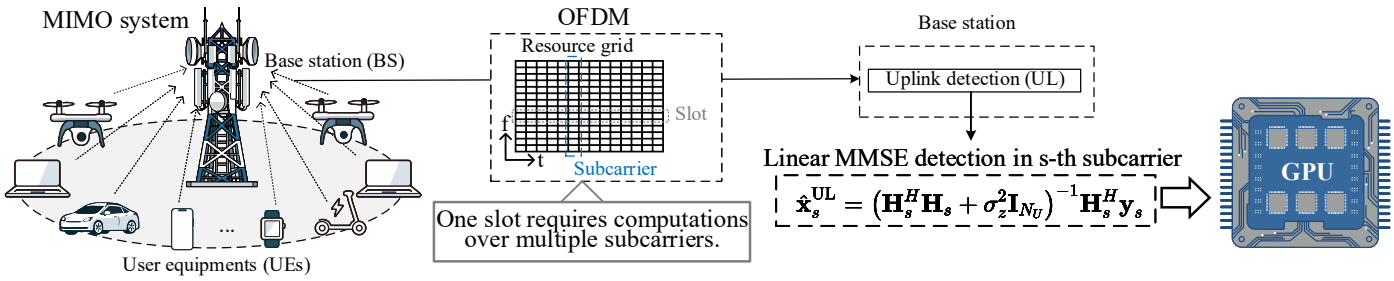
This section presents the mathematical models used in our study. We first describe the OFDM (Orthogonal Frequency-Division Multiplexing)-MIMO uplink system model, then present the linear MMSE detection. Finally, we review the Neumann series approximation as a general approach to approximate the required matrix inversion.

A. System Model

As shown in Fig. 1, we consider an OFDM-MIMO system with S subcarriers, where the BS is equipped with N_B antennas serving N_U single-antenna users. The uplink transmissions are assumed perfectly synchronized. The channel is modeled as flat Rayleigh fading, with each entry independently drawn from a complex Gaussian distribution with zero mean and unit variance, i.e., $\mathcal{CN}(0, 1)$. Additive white Gaussian noise (AWGN) is present at both the BS and the users, with variance N_0.

The received signal $\mathbf{y}_s \in \mathbb{C}^{N_B \times 1}$ at the BS for the s-th subcarrier is modeled as

$$\mathbf{y}_s = \mathbf{H}_s \mathbf{x}_s + \mathbf{z}_s, \tag{1}$$



(a) MIMO-OFDM System (b) Accelerating Target Matrix Computation using GPU

Fig. 1: GPU-accelerated architecture for massive MIMO-OFDM systems. The heavy computational workloads of MIMO transceiver tasks, which are generated across numerous OFDM subcarriers (a), are offloaded to a GPU (b) to efficiently execute complex matrix operations.

where $\mathbf{x}_s \in \mathbb{C}^{N_U \times 1}$ is the user transmit vector, normalized as $\mathbb{E}\{\mathbf{x}_s(\mathbf{x}_s)^{\mathrm{H}}\} = \mathbf{I}_{N_U}$, $\mathbf{H}_s \in \mathbb{C}^{N_B \times N_U}$ is the channel matrix, and $\mathbf{z}_s \in \mathbb{C}^{N_B \times 1}$ is the AWGN vector at the BS.

B. Linear MMSE Signal Detection

In the uplink scenario, the BS is responsible for detecting the data streams transmitted simultaneously by multiple users. Specifically, the multi-user detector at the BS computes the a posteriori log-likelihood ratios of the bits associated with the modulated symbols. Given the channel matrix $\mathbf{H}_s$, the linear MMSE detector for the s-th subcarrier can be expressed as

$$\hat{\mathbf{x}}_s = \left(\mathbf{H}_s^{\mathrm{H}} \mathbf{H}_s + \sigma_z^2 \mathbf{I}_{N_U}\right)^{-1} \mathbf{H}_s^{\mathrm{H}} \mathbf{y}_s. \tag{2}$$

The matrix to be inverted can be expressed as

$$\mathbf{W}_s = \left(\mathbf{H}_s^{\mathrm{H}} \mathbf{H}_s + \sigma_z^2 \mathbf{I}_{N_U}\right). \tag{3}$$

C. Neumann Series Approximation

The computational cost of computation of direct inversion $\mathbf{W}_s$ is high. To address this, we introduce the Neumann series method, which approximates the inverse of $\mathbf{W}_s$. The Neumann series expansion uses an iterative form to improve accuracy step by step, and can reduce the high computational and power cost of exact decomposition methods. Specifically, when $\mathbf{W}_s = \mathbf{I} - \mathbf{A}$, its inverse can be expanded as

$$\mathbf{W}_s^{-1} = (\mathbf{I} - \mathbf{A})^{-1} = \sum_{l=0}^{\infty} \mathbf{A}^l. \tag{4}$$

For $\mathbf{W}_s = \mathbf{X}_s + \mathbf{E}_s$, where $\mathbf{X}_s$ is the seed matrix and $\mathbf{E}_s$ is the residual matrix, the inverse can be expressed as

$$\mathbf{W}_s^{-1} = \mathbf{X}_s^{-1} \sum_{l=0}^{\infty} \left(-\mathbf{E}_s \mathbf{X}_s^{-1}\right)^l. \tag{5}$$

In practice, the infinite sum is truncated to L terms to reduce complexity. The resulting approximation, denoted by $\mathbf{W}_{s,L}^{-1}$, is given by

$$\mathbf{W}_{s,L}^{-1} = \mathbf{X}_s^{-1} \sum_{l=0}^{L-1} (-\mathbf{E}_s \mathbf{X}_s^{-1})^l. \tag{6}$$

IV. A Novel Design For Neumann Series Approximate Matrix Inversion

This section presents a novel Neumann-series–based approximate matrix inversion framework. By applying block-diagonal partitioning, the method improves convergence across various antenna ratios. A hierarchical GPU-parallel algorithm is developed to exploit SIMD (Single Instruction, Multiple Data) hardware, while shared-memory optimization, preallocated matrix space, and half-precision arithmetic further enhance computational efficiency.

A. Design of Block-Diagonal Neumann Seeds

1) Block-Diagonal Structure. In the Neumann series expansion, the choice of the seed matrix $\mathbf{X}_s$ is crucial for both computational efficiency and convergence. In this work, we propose a Neumann seed selection strategy based on a block-diagonal partition of the target matrix $\mathbf{W}_s$. Specifically, $\mathbf{W}_s$ is divided into N equal-sized square blocks, where $N = k^2$ as both the row and column dimensions are divided into k segments. The seed matrix $\mathbf{X}_s$ is then constructed by retaining the diagonal blocks, while the off-diagonal blocks are assigned to the residual matrix $\mathbf{E}_s$. Formally, this decomposition can be expressed as

$$\mathbf{W}_s = \begin{bmatrix} \mathbf{W}_{s,11} & \mathbf{W}_{s,12} & \cdots & \mathbf{W}_{s,1k} \\ \mathbf{W}_{s,21} & \mathbf{W}_{s,22} & \cdots & \mathbf{W}_{s,2k} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{W}_{s,k1} & \mathbf{W}_{s,k2} & \cdots & \mathbf{W}_{s,kk} \end{bmatrix}, \tag{7}$$

with the block-diagonal seed and the residual defined as

$$\mathbf{X}_s = \begin{bmatrix} \mathbf{W}_{s,11} & 0 & \cdots & 0 \\ 0 & \mathbf{W}_{s,22} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \mathbf{W}_{s,kk} \end{bmatrix}, \quad \mathbf{E}_s = \mathbf{W}_s - \mathbf{X}_s. \tag{8}$$

The motivation lies in the mathematical tractability of block-diagonal matrices. The block-diagonal structure of $\mathbf{X}_s$ guaran-

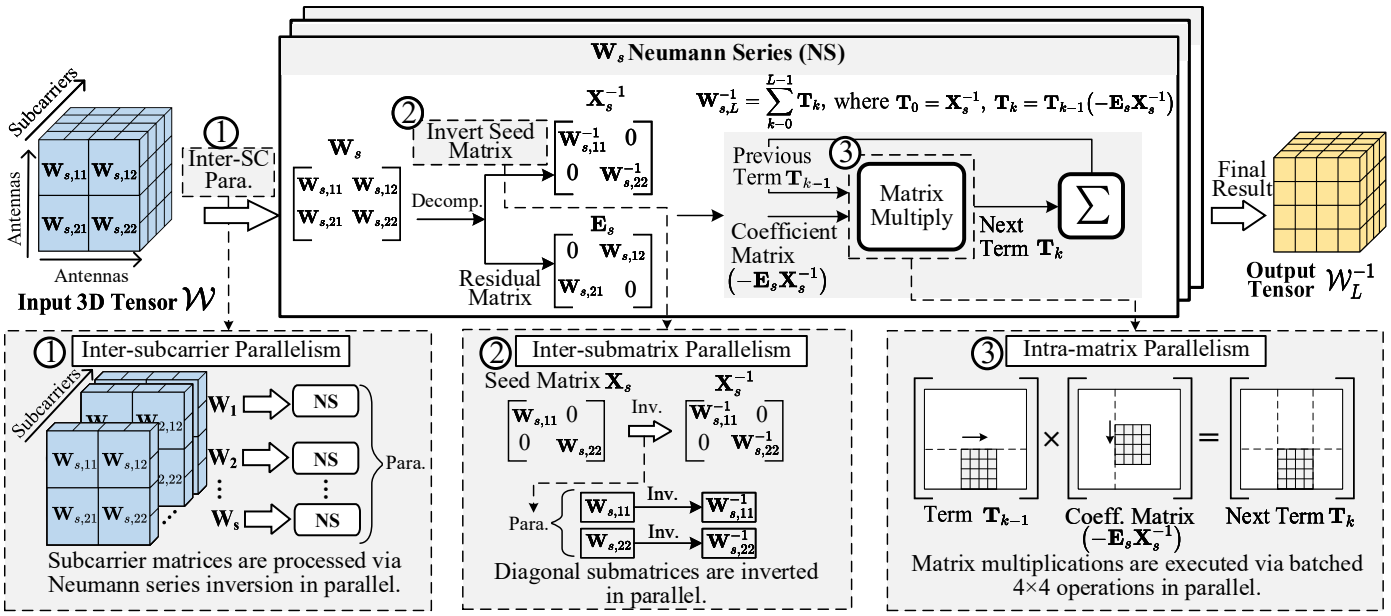


Fig. 2: Hierarchical parallel framework of ParaNeumann across inter-subcarrier, inter-submatrix, and intra-matrix levels.

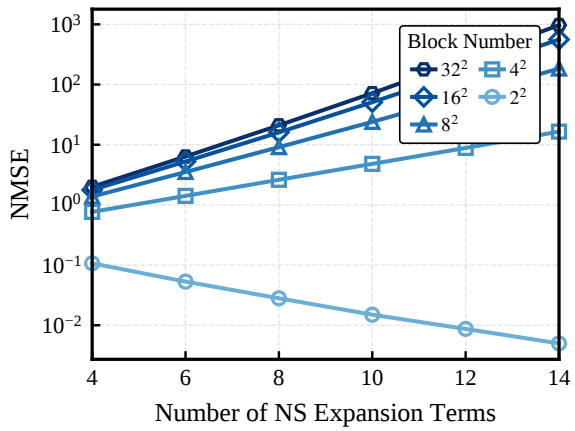


Fig. 3: NMSE of approximate matrix inverse under different partitioning strategies. Only the 4-blocks partition achieves consistent inversion error reduction as the number of Neumann series terms increases, while other schemes fail to effectively reduce the inversion error.

tees that its invertibility is equivalent to the invertibility of its diagonal blocks. More formally, for

$$\mathbf{X}_s = \mathrm{diag}(\mathbf{W}_{s,11}, \mathbf{W}_{s,22}, \ldots, \mathbf{W}_{s,kk}), \tag{9}$$

if all $\mathbf{X}_{s,i}$ are invertible, then

$$\mathbf{X}_s^{-1} = \mathrm{diag}(\mathbf{W}_{s,11}^{-1}, \mathbf{W}_{s,22}^{-1}, \ldots, \mathbf{W}_{s,kk}^{-1}). \tag{10}$$

This property allows each block to be inverted independently, enabling direct assembly of the global inverse $\mathbf{X}_s^{-1}$ without additional correction terms.

2) Partition Size Selection. A further important question is how many blocks should be used in practice. To evaluate this, we conduct an experiment in which $\mathbf{W}_s$ is partitioned into $N = k^2$ equal-sized blocks. For each configuration, the accuracy of the approximate inverse is quantified by the normalized mean-square error (NMSE) between the approximate and exact inverses. All experiments are conducted with an antenna ratio of $r = 1$.

As shown in Fig. 3, each curve corresponds to a distinct partition configuration. The results indicate that only the configuration with four blocks achieves stable convergence, where the inversion error consistently decreases as the series order L increases. In contrast, other block-number schemes lead to divergence, where more expansion terms actually worsen the approximation.

3) Optimized Matrix Multiplication Using Block Structure. As shown in (6), the Neumann expansion involves recursive multiplications of $(-\mathbf{X}_s^{-1}\mathbf{E}_s)^l$, which dominate the computational cost. To reduce complexity, we exploit the block-diagonal and block anti-diagonal structures of $\mathbf{X}_s$ and $\mathbf{E}_s$, yielding

$$\mathbf{X}_s^{-1}\mathbf{E}_s = \begin{bmatrix} 0 & \mathbf{W}_{s,11}^{-1}\mathbf{W}_{s,12} \\ \mathbf{W}_{s,22}^{-1}\mathbf{W}_{s,21} & 0 \end{bmatrix}. \tag{11}$$

The result remains block anti-diagonal, allowing computations only on nonzero blocks. This structural optimization eliminates redundant operations, reducing complexity from $O(2N^3)$ to $O(N^3/2)$ and memory cost by storing only diagonal and anti-diagonal components.

B. GPU Parallelization and Memory-Efficient Implementation.

To translate the structural advantages of ParaNeumann into high throughput, we adopt an algorithm-hardware co-design approach that bridges the algorithmic structure and GPU execution characteristics. Guided by this principle, we construct

a hierarchical parallel framework to fully exploit the massive parallelism of GPUs. In addition, memory access and data reuse strategies are carefully optimized to sustain high computational throughput and alleviate bandwidth bottlenecks.

1) Hierarchical Parallelism. To efficiently map ParaNeumann onto the GPU, we design a structure-guided parallelization framework that identifies independent and batch-parallel computation tasks that fit the GPU's SIMD execution model, enabling a hierarchical parallel structure, as shown in Fig. 2.

(i) Inter-subcarrier parallelism. In an OFDM-MIMO system, the matrices corresponding to different subcarriers are completely independent. This independence allows the inversion of $\mathbf{W}_s$ at each subcarrier to be treated as an isolated task. Since no data dependencies exist across subcarriers, these tasks can be distributed across SIMD units and executed in parallel, resulting in a significant increase in throughput.

(ii) Inter-submatrix parallelism. The block-diagonal structure of $\mathbf{X}_s$ ensures that its diagonal submatrices are fully decoupled from one another according to (10). As a result, each diagonal block can be inverted independently without requiring inter-block communication or correction terms. This decomposition converts the large inversion problem into a set of smaller, independent inverse computations, each of which is well suited for SIMD execution.

(iii) Intra-matrix parallelism. Neumann series expansion involves matrix multiplications of the form $(\mathbf{E}_s\mathbf{X}_s^{-1})^l$. These operations are inherently parallelizable because each output element is computed as a dot product between rows and columns. We further enhance this step by exploiting GPU Tensor Cores, which execute fused MMA operations on 4×4 tiles in a single clock cycle. By reformulating the Neumann series updates into sequences of tile-based operations, the algorithm achieves a dramatic increase in arithmetic throughput.

2) Memory Optimization. In addition to parallelization, memory access patterns critically affect GPU performance. Although modern GPU provides massive computational throughput, inefficient memory usage can easily become the bottleneck. To alleviate this issue, we adopt two complementary strategies that are naturally aligned with the structure of ParaNeumann.

(i) Reuse of intermediate results. During the Neumann series iterations, intermediate terms such as $(\mathbf{E}_s\mathbf{X}_s^{-1})^l$ are repeatedly involved in subsequent multiplications. Instead of recomputing or redundantly fetching these terms from global memory, we organize the computation to reuse the intermediate results in a persistent memory buffer. This reduces redundant memory transactions and ensures that frequently accessed data remains close to the GPU cores, improving effective memory bandwidth utilization.

(ii) Preallocation of working space. Memory allocation and deallocation on the GPU are expensive operations that may cause synchronization overhead. To avoid this, we preallocate contiguous memory regions to store all intermediate matrices required in the Neumann expansion. By reusing this preallocated space across iterations, we minimize allocator pressure and maintain memory alignment, both of which contribute to higher data throughput.

V. Evaluation

We evaluate the proposed ParaNeumann from two aspects: inversion accuracy and computational efficiency. Accuracy is measured by BER in detection, while efficiency is evaluated by algorithm runtime.

A. Experimental Setup

1) Evaluation Environment Setup. Experiments are conducted on an NVIDIA RTX 4090 GPU (CUDA 12.3, 24 GB, compute capability 8.9) with an AMD Ryzen Threadripper PRO 5975WX CPU under Linux 6.8. Perfect channel state information and ideal OFDM subcarrier orthogonality are assumed, excluding channel estimation and inter-carrier interference from the evaluation scope.

2) Comparison Methods. To evaluate the performance of the proposed ParaNeumann framework, we compare it against two following representative matrix inversion methods.

LU Decomposition-based Exact Inversion: This method serves as the *gold standard* for exact matrix inversion, providing a numerically stable and precise solution by solving systems of linear equations. Since it yields the exact solution, LU decomposition represents the theoretical performance upper bound for linear detection and precoding schemes. To ensure a rigorous and competitive efficiency comparison, our implementation leverages the NVIDIA cuBLAS library [25]. As an industrial-grade, state-of-the-art (SOTA) BLAS implementation, cuBLAS features highly-engineered parallelization and memory optimization strategies that fully exploit GPU hardware potential.

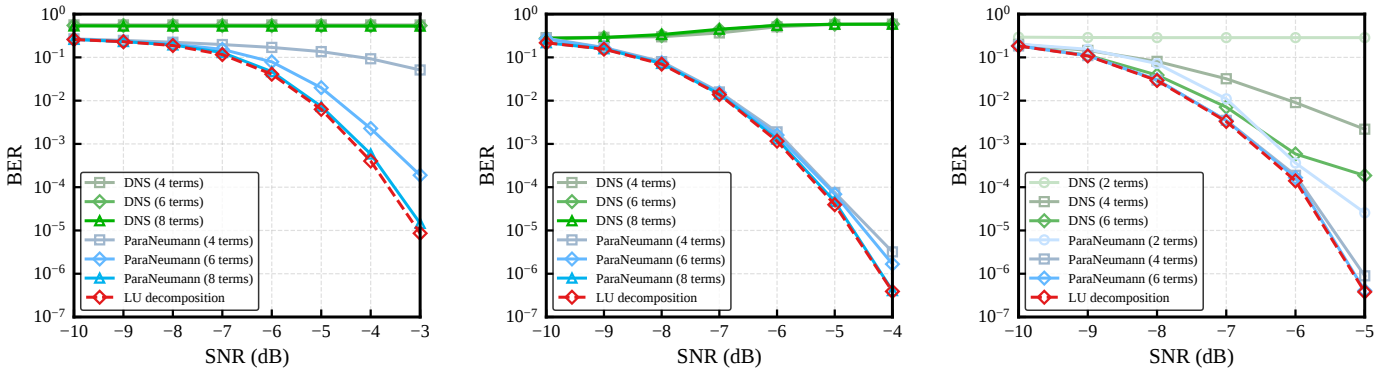
Diagonal-based Neumann Series (DNS) Approximation: As a widely adopted approximate inversion technique, DNS selects the diagonal elements of $\mathbf{W}_s$ as the initial Neumann seed. For a fair and comprehensive comparison, our DNS implementation is also highly optimized for GPU hardware, utilizing efficient parallel kernels to ensure it reflects the best possible performance for diagonal-based approximation on modern GPUs. This allows us to evaluate the specific gains in convergence and parallel efficiency provided by our proposed block-diagonal architecture.

B. BER Performance

We evaluate inversion performance using the BER, defined as the ratio of incorrectly detected bits to total transmitted bits. Experiments are conducted under both asymmetric ($r > 1$) and symmetric ($r = 1$) MIMO settings.

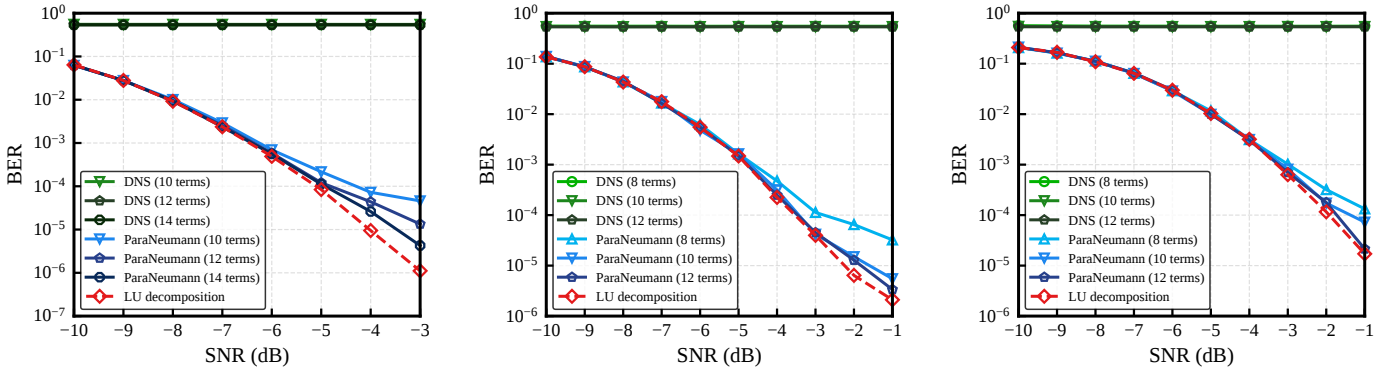
1) Asymmetric MIMO. We consider a BS with 1024 antennas serving different numbers of users, with ratios r from 2 to 8, 64-QAM modulation, and Low-Density Parity-Check Code (LDPC) coding rate of 0.5. As shown in Fig. 4, ParaNeumann achieves BER performance comparable to LU decomposition while significantly outperforming the DNS, especially at low antenna ratios.

2) Symmetric MIMO. For $r = 1$, we test three configurations: 1024×1024, 512×512, and 256×256, using 4-QAM



(a) 1024×512 MIMO. (b) 1024×256 MIMO. (c) 1024×128 MIMO.

Fig. 4: BER performance comparison in asymmetric MIMO scenarios. With sufficient expansion terms, the proposed ParaNeumann method closely approximates the BER performance of LU decomposition in MMSE detection, while DNS fails to match LU decomposition performance except when the antenna ratio r is high.



(a) 1024×1024 MIMO. (b) 512×512 MIMO. (c) 256×256 MIMO.

Fig. 5: BER performance comparison in symmetric MIMO antenna scenarios. With sufficient expansion terms, ParaNeumann remains effective and approximates the BER performance of LU decomposition, while DNS exhibits poor BER performance.

modulation under the same system setup. As shown in Fig. 5, ParaNeumann achieves BER performance within 0.5 dB of LU decomposition when sufficient Neumann terms are applied, while remaining significantly superior to DNS.

C. Compute Efficiency

In this part, we evaluate the computational efficiency of the proposed ParaNeumann in comparison to LU decomposition and DNS, focusing on hardware acceleration gains. We assess its performance through execution time.

We compare runtime performance using 10 Neumann terms for both ParaNeumann and DNS. The inverse target is represented as a 3D tensor $\mathcal{W} \in \mathbb{C}^{S \times N_U \times N_U}$, and each test is averaged over multiple runs to mitigate computation noise. As shown in Fig. 6, ParaNeumann achieves the lowest runtime. Specifically, for $(512, 512, 512)$ tensors, its execution time is only one-tenth that of DNS and one-third that of LU decomposition.

The above results confirm that the proposed ParaNeumann framework enables highly efficient approximate matrix inversion, achieving BER performance close to LU decomposition while requiring significantly less computation time compared with both LU decomposition and DNS algorithms.

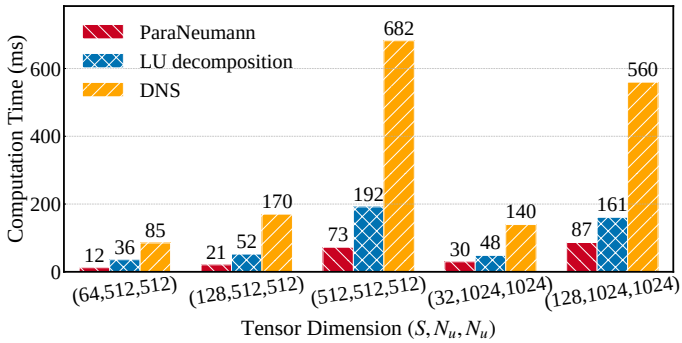


Fig. 6: Computation time comparison. ParaNeumann reduces the computation time to approximately one-tenth compared with DNS, while requiring only one-third the time of LU decomposition.

VI. Conclusion

This paper presents ParaNeumann, a GPU-accelerated hierarchical parallel framework for matrix inversion that achieves near-real-time computation and flexible scalability in large-scale MIMO detection. While introducing additional algorithmic complexity, its high degree of parallelism significantly boosts computational throughput. Implementation results on a commercial GPU demonstrate that our framework strikes a favorable balance between communication performance and processing efficiency. Future work will explore the efficient parallelization of computationally intensive factorizations, such as SVD, to further advance the practical deployment of massive MIMO systems.

References

[1] ITU-R, "Framework and Overall Objectives of the Future Development of IMT for 2030 and Beyond," ITU-R, Tech. Rep., Jun 2023, accessed: Sep. 26, 2025. [Online]. Available: https://www.itu.int/en/ITU-R/study-groups/rsg5/rwp5d/imt-2030/pages/default.aspx/

[2] Z. Wang, J. Zhang, H. Du, D. Niyato, S. Cui, B. Ai, M. Debbah, K. B. Letaief, and H. V. Poor, "A tutorial on extremely large-scale MIMO for 6G: Fundamentals, signal processing, and applications," *IEEE Commun. Surveys Tuts.*, vol. 26, no. 3, pp. 1560–1605, 2024.

[3] H. Wang, Y. Ji, Y. Shen, W. Song, M. Li, X. You, and C. Zhang, "An efficient detector for massive MIMO based on improved matrix partition," *IEEE Trans. Signal Process.*, vol. 69, pp. 2971–2986, 2021.

[4] NVIDIA, "NVIDIA Aerial CUDA-Accelerated RAN," 2025, accessed: Sep. 26, 2025. [Online]. Available: https://developer.nvidia.com/aerial-cuda-accelerated-ran/

[5] R. H. Bartels and G. H. Golub, "The simplex method of linear programming using LU decomposition," *ACM Commun.*, vol. 12, no. 5, pp. 266–268, 1969.

[6] D. Zhu, B. Li, and P. Liang, "On the matrix inversion approximation based on Neumann series in massive MIMO systems," in *Proc. IEEE ICC*, 2015, pp. 1763–1769.

[7] F. Jiang, C. Li, Z. Gong, and R. Su, "Stair matrix and its applications to massive MIMO uplink data detection," *IEEE Trans. Commun.*, vol. 66, no. 6, pp. 2437–2455, 2018.

[8] F. Jiang, C. Li, and Z. Gong, "Low complexity and fast processing algorithms for V2I massive MIMO uplink detection," *IEEE Trans. Veh. Technol.*, vol. 67, no. 6, pp. 5054–5068, 2018.

[9] Y. Chen, Y. T. Hou, W. Lou, J. H. Reed, and S. Kompella, "M3: A sub-millisecond scheduler for multi-cell mimo networks under c-ran architecture," in *Proc. IEEE INFOCOM*, 2022, pp. 130–139.

[10] A. Dabah, Z. Rezki, H. Ltaief, D. Keyes, and M.-S. Alouini, "Efficient gpu-based large mimo detection algorithm for next-generation communication systems," in *Proc. IEEE GLOBECOM*, 2023, pp. 3372–3377.

[11] C. M. Jozsa, G. Kolumban, A. M. Vidal, F.-J. Martinez-Zaldivar, and A. Gonzalez, "New parallel sphere detector algorithm providing high-throughput for optimal MIMO detection," *Elsevier Procedia Computer Science*, vol. 18, pp. 2432–2435, 2013.

[12] B. Gokalgandhi, J. Ling, Z. Latinovic, D. Samardzija, and I. Seskar, "Parallel implementation of K-best quadrature amplitude modulation detection for massive multiple-input multiple-output systems," *MDPI Electronics*, vol. 13, no. 14, p. 2775, 2024.

[13] S. Roger, C. Ramiro, A. Gonzalez, V. Almenar, and A. M. Vidal, "An efficient GPU implementation of fixed-complexity sphere decoders for MIMO wireless systems," *Integrated Computer-Aided Engineering*, vol. 19, no. 4, pp. 341–350, 2012.

[14] D. Sui, Y. Li, J. Wang, P. Wang, and B. Zhou, "High throughput MIMO-OFDM detection with graphics processing units," in *Proc. IEEE CSAE*, vol. 2, 2012, pp. 176–179.

[15] K. Li, B. Yin, M. Wu, J. R. Cavallaro, and C. Studer, "Accelerating massive MIMO uplink detection on GPU for SDR systems," in *Proc. IEEE DCAS*, 2015, pp. 1–4.

[16] L. Dai, X. Gao, X. Su, S. Han, Z. Wang *et al.*, "Low-complexity soft-output signal detection based on Gauss-Seidel method for uplink multiuser large-scale MIMO systems," *IEEE Trans. Veh. Technol.*, vol. 64, no. 10, pp. 4839–4845, 2015.

[17] B. Y. Kong and I.-C. Park, "Low-complexity symbol detection for massive MIMO uplink based on Jacobi method," in *Proc. IEEE Int. Symp. Pers. Indoor, Mobile Radio Commun.*, 2016, pp. 1–5.

[18] V. Gupta, A. K. Sah, and A. K. Chaturvedi, "Iterative matrix inversion based low complexity detection in large/massive MIMO systems," in *Proc. IEEE ICC*, 2016, pp. 712–717.

[19] K. Nikitopoulos, D. Chatzipanagiotis, C. Jayawardena, and R. Tafazolli, "Multisphere: Massively parallel tree search for large sphere decoders," in *IEEE GLOBECOM*, 2016, pp. 1–6.

[20] K. Nikitopoulos, G. Georgis, C. Jayawardena, D. Chatzipanagiotis, and R. Tafazolli, "Massively parallel tree search for high-dimensional sphere decoders," *IEEE Trans. Parallel Distrib. Syst.*, vol. 30, no. 10, pp. 2309–2325, 2018.

[21] C. Jayawardena and K. Nikitopoulos, "G-MultiSphere: Generalizing massively parallel detection for non-orthogonal signal transmissions," *IEEE Trans. Commun.*, vol. 68, no. 2, pp. 1227–1239, 2019.

[22] C. Husmann, G. Georgis, K. Nikitopoulos, and K. Jamieson, "FlexCore: Massively parallel and flexible processing for large MIMO access points," in *USENIX NSDI*, 2017, pp. 197–211.

[23] M. Matthaiou, M. R. McKay, P. J. Smith, and J. A. Nossek, "On the condition number distribution of complex Wishart matrices," *IEEE Trans. Commun.*, vol. 58, no. 6, pp. 1705–1717, 2010.

[24] F. Jiang, C. Li, and Z. Gong, "Block Gauss-Seidel method based detection in vehicle-to-infrastructure massive MIMO uplink," in *Proc. IEEE GLOBECOM*, 2017, pp. 1–6.

[25] NVIDIA Corporation, "NVIDIA cuBLAS Library," 2024, accessed: Mar. 11, 2025. [Online]. Available: https://docs.nvidia.com/cuda/cublas/